

Name bhavya rana

Sap id 590034689

Batch 15

1) Column-by-column input and row-major output using a single loop

Algorithm

1. Start.
2. Declare integers `rows` and `cols`.
3. Read `rows`.
4. If `rows` ≤ 0 , display an error and repeat until a positive value is entered.
5. Read `cols`.
6. If `cols` ≤ 0 , display an error and repeat until a positive value is entered.
7. Declare a variable-length 2D integer array `a[rows][cols]`.
8. Read the elements **column by column**:
 - Set `j = 0`.
 - Repeat while `j < cols`.
 - For each column, set `i = 0`.
 - Repeat while `i < rows`.
 - Read `a[i][j]`.
9. Set a linear index `k = 0`.
10. Repeat while `k < rows * cols`:
 - Row index = `k / cols`.
 - Column index = `k % cols`.
 - Print `a[k / cols][k % cols]`.
 - If `(k + 1) % cols == 0`, print a newline.
11. Stop.

C Program

```
#include <stdio.h>
```

```
int main()  
{
```

```
int rows, cols;
int i, j, k;

/* Read and validate number of rows */
do
{
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    if (rows <= 0)
    {
        printf("Error: Number of rows must be positive.
\n");
    }

} while (rows <= 0);

/* Read and validate number of columns */
do
{
    printf("Enter the number of columns: ");
    scanf("%d", &cols);

    if (cols <= 0)
    {
        printf("Error: Number of columns must be
positive.\n");
    }

} while (cols <= 0);

/* Variable-length 2D array */
int a[rows][cols];

/* Read elements column by column */
printf("\nEnter the array elements column by column:\n");

for (j = 0; j < cols; j++)
{
    for (i = 0; i < rows; i++)
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
```

```
/* Display in row-major order using one loop */
printf("\nArray elements in row-major order:\n");

for (k = 0; k < rows * cols; k++)
{
    printf("%d ", a[k / cols][k % cols]);

    if ((k + 1) % cols == 0)
    {
        printf("\n");
    }
}

return 0;
}
```

Execution

```
[bhavya@Bhavyas-MacBook-Air ~ %
[bhavya@Bhavyas-MacBook-Air ~ % nano day28.cpp
[bhavya@Bhavyas-MacBook-Air ~ % clang day28.cpp -o day28
[bhavya@Bhavyas-MacBook-Air ~ % ./day28
Enter the number of rows: 2
Enter the number of columns: 3

Enter the array elements column by column:
a[0][0] = 1
a[1][0] = 4
a[0][1] = 2
a[1][1] = 5
a[0][2] = 3
a[1][2] = 6

Array elements in row-major order:
1 2 3
4 5 6
bhavya@Bhavyas-MacBook-Air ~ %
```

Test-Case Table

Test Case	Input	Expected Output	Result
1 – Valid	rows=2, cols=3; elements: 1,4,2,5,3,6	1 2 3 / 4 5 6	Pass
2 – Boundary	rows=1, cols=1; element: 10	10	Pass
3 – Invalid rows	rows=0, then 2; cols=2	Error for rows=0, then accepts 2	Pass
4 – Invalid columns	rows=2; cols=-3, then 2	Error for cols=-3, then accepts 2	Pass
5 – Both initially invalid	rows=-2, then 1; cols=0, then 3	Repeats both invalid inputs	Pass

2) Copy a variable-length 2D array into a 1D array in row-major order

Algorithm

1. Start.
2. Declare `rows` and `cols`.
3. Read and validate `rows` until it is positive.
4. Read and validate `cols` until it is positive.
5. Declare the VLA `a[rows][cols]`.
6. Declare the 1D VLA `b[rows * cols]`.
7. Set `k = 0`.
8. Using nested loops, read each element of the 2D array in row-major order.
9. Copy each element into `b[k]` and increment `k`.
10. Display the 2D array using nested loops.
11. Display the 1D array using a single loop.
12. Stop.

C Program

```
#include <stdio.h>

int main()
{
    int rows, cols;
    int i, j, k = 0;

    /* Read and validate rows */
    do
```

```
{
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    if (rows <= 0)
    {
        printf("Error: Number of rows must be positive.
\n");
    }

    while (rows <= 0);

    /* Read and validate columns */
    do
    {
        printf("Enter the number of columns: ");
        scanf("%d", &cols);

        if (cols <= 0)
        {
            printf("Error: Number of columns must be
positive.\n");
        }

        while (cols <= 0);

        /* Variable-length arrays */
        int a[rows][cols];
        int b[rows * cols];

        /* Read elements into the 2D array */
        printf("\nEnter the elements of the 2D array:\n");

        for (i = 0; i < rows; i++)
        {
            for (j = 0; j < cols; j++)
            {
                printf("a[%d][%d] = ", i, j);
                scanf("%d", &a[i][j]);
            }
        }

        /* Copy into 1D array in row-major order */
        k = 0;
```

```
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            b[k] = a[i][j];
            k++;
        }
    }

    /* Display 2D array */
    printf("\n2D Array:\n");

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("%d ", a[i][j]);
        }

        printf("\n");
    }

    /* Display 1D array */
    printf("\n1D Array in row-major order:\n");

    for (k = 0; k < rows * cols; k++)
    {
        printf("%d ", b[k]);
    }

    printf("\n");

    return 0;
}
```

Execution

```

Last login: Mon Sep 21 23:09:23 on ttys000
[bhavya@Bhavyas-MacBook-Air ~ % nano day28.cp
[bhavya@Bhavyas-MacBook-Air ~ % clang day28.cp -o day28
[bhavya@Bhavyas-MacBook-Air ~ % ./day28
Enter the number of rows: 2
Enter the number of columns: 3

Enter the elements of the 2D array:
a[0][0] = 10
a[0][1] = 20
a[0][2] = 30
a[1][0] = 40
a[1][1] = 50
a[1][2] = 60

2D Array:
10 20 30
40 50 60

1D Array in row-major order:
10 20 30 40 50 60
bhavya@Bhavyas-MacBook-Air ~ % █

```

Test- Case Table

Test Case	Rows × Columns	Elements	Expected 1D Array	Result
1 – Valid	2 × 3	1 2 3 / 4 5 6	1 2 3 4 5 6	Pass
2 – Square	3 × 3	1 to 9	1 2 3 4 5 6 7 8 9	Pass
3 – Boundary	1 × 1	25	25	Pass
4 – Single row	1 × 4	10 20 30 40	10 20 30 40	Pass
5 – Single	4 × 1	5 10 15 20	5 10 15 20	Pass
6 – Invalid dimensions	rows=0 then 2, cols=-1 then 2	1 2 3 4	Reject invalid dimensions, then output 1 2 3 4	Pass

Important concept: In Program 1, k / cols determines the **row** and $k \% \text{cols}$ determines the **column**. Therefore,

`a[k / cols][k % cols]`

allows a 2D array to be traversed in **row-major order with only one loop**, as required.

